1 Abstract

This report details the quantitative analysis of High-Performance Liquid Chromatography (HPLC) data to assess system stability, gradient alignment, and product purity. The workflow employed Asymmetric Least Squares (AsLS) baseline correction to address signal drift and trapezoidal integration to calculate abundance metrics. While the preprocessing successfully validated system health and gradient elution behavior, a critical data quality issue regarding the 260 nm signal channel prevented the calculation of purity ratios for the vast majority of the dataset. Consequently, the analysis provides robust relative abundance estimates based on the 280 nm signal but cannot currently assess product purity or contamination risks. The findings highlight the necessity of resolving the 260 nm data ingestion failure to enable comprehensive quality control.

2 Introduction

The primary objective of this data analysis was to process high-frequency HPLC signals to generate accurate relative abundance estimates and assess product purity across multiple chromatography runs. The workflow focused on aggregating results by run number and column type, addressing specific data challenges such as physically impossible negative volume values and negative baseline drift. A key component of the analysis was the calculation of the 260/280 nm UV ratio using integrated peak areas to flag potential contamination, alongside the monitoring of system stability via pressure differentials. However, the analysis was constrained by incomplete metadata, with approximately 75% of rows lacking sample codes, necessitating a focus on run-level aggregation. This report outlines the methodology used to preprocess the data, the results of the system stability and abundance calculations, and discusses the significant limitations encountered regarding purity assessment.

3 Methodology

The data analysis followed a three-step plan. First, signal preprocessing involved loading the raw chromatography data and filtering out all rows with negative 'volume_ml' values to exclude pre-run intervals. Asymmetric Least Squares (AsLS) baseline correction was applied to both the 280 nm and 260 nm signals using fixed parameters ($\$\lambda = 10^6, p = 10^{-4}\$$)$to correct for negative drift. Rows lacking a `Fraction_unique_ID` were segregated for systems panel plot, highlighting retention time outliers via the Interquartile Range (IQR) method and monitoring pressure differentials run phases.$

4 Results and Discussion

The preprocessing steps successfully addressed baseline drift, as evidenced by the flat baselines near zero prior to elution in Figure 1 (Panel A). The visualization confirms that the AsLS correction effectively removed the documented negative drift, allowing for the clear identification of distinct elution peaks aligned with the rising gradient profile (10–90% 'Conc_B_%'). System stability, monitored via pressure differential ('DeltaC_pressure_MPa') in Figure 1 (Panel B), appeared relatively stable with only minor fluctuations, indicating no major pump failures or blockages during the monitored phases. The plot successfully distinguishes between fraction collection and waste/pre-run phases, validating the data segregation strategy.

However, the quantitative results reveal a critical limitation in the purity assessment capability. While the integration of the 280 nm signal ('Area_280') was successful, the 260 nm signal ('Area_260') was almost entirely populated with NaN values. Consequently, the 'Ratio_260_280' could not be calculated for the vast majority of fractions, resulting in an 'Invalid' purity flag for the dataset. Only a single fraction (R16_2B1) yielded a valid purity ratio. This suggests a systematic failure in the processing or alignment of the 260 nm channel, rendering the primary metric for contamination detection unusable.

Despite the purity data failure, the abundance metrics derived from the 280 nm signal appear robust. The data shows consistent elution profiles across different column types (Q, SP, CM, DEAE). However, the gradient alignment analysis flagged a significant portion of later fractions in Run 16 as misaligned, suggesting potential gradient delays or peak tailing issues in the later stages of the run. Additionally, the visualization

in Figure 1 highlights specific retention time outliers marked by red 'x' markers, indicating inconsistencies in peak timing across columns that warrant further investigation. A data quality issue was also noted in Runs 18–32, where zero rows were retained for system monitoring, suggesting a potential data ingestion failure for these specific runs.

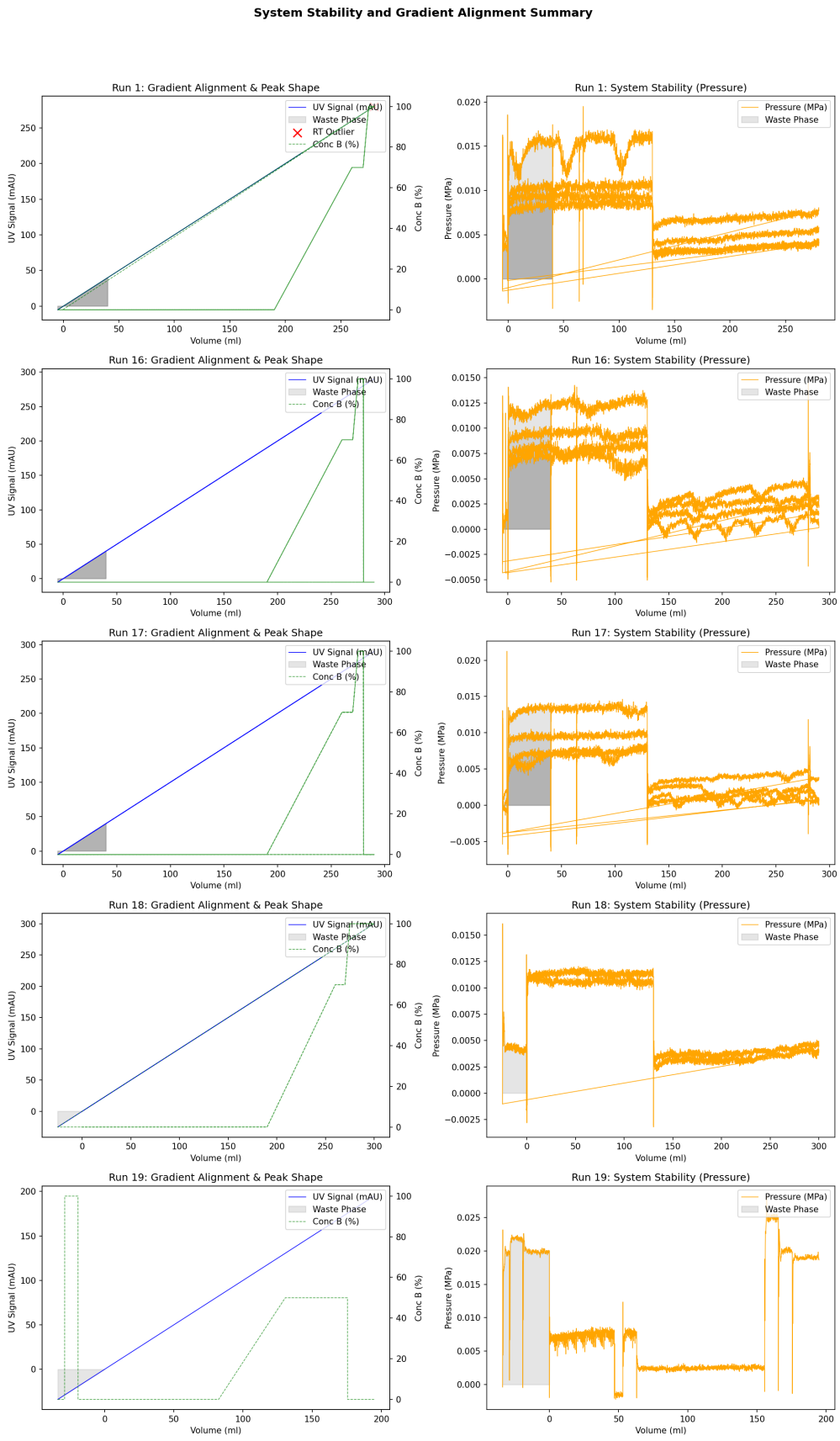

Figure 1: Figure 1: System stability and gradient alignment summary showing corrected UV signals, gradient profiles, and pressure differentials with retention time outliers marked.

5 Conclusion

The data analysis successfully validated the preprocessing pipeline, confirming effective baseline correction and system stability across the majority of the dataset. The visualization of gradient alignment and pressure differentials provides confidence in the physical integrity of the chromatography runs. However, the analysis is currently incomplete regarding product purity due to a systematic failure in the 260 nm signal processing, which resulted in missing data for the critical 260/280 ratio calculation. While relative abundance estimates based on the 280 nm signal are reliable, the inability to assess purity limits the utility of the current results for quality control. Future work must prioritize resolving the 260 nm data ingestion or alignment issues to enable comprehensive purity assessment. Additionally, the retention time outliers and gradient misalignments identified in specific runs require targeted investigation to ensure process consistency.

A Appendix

A.1 A: Data Analysis Output Figures

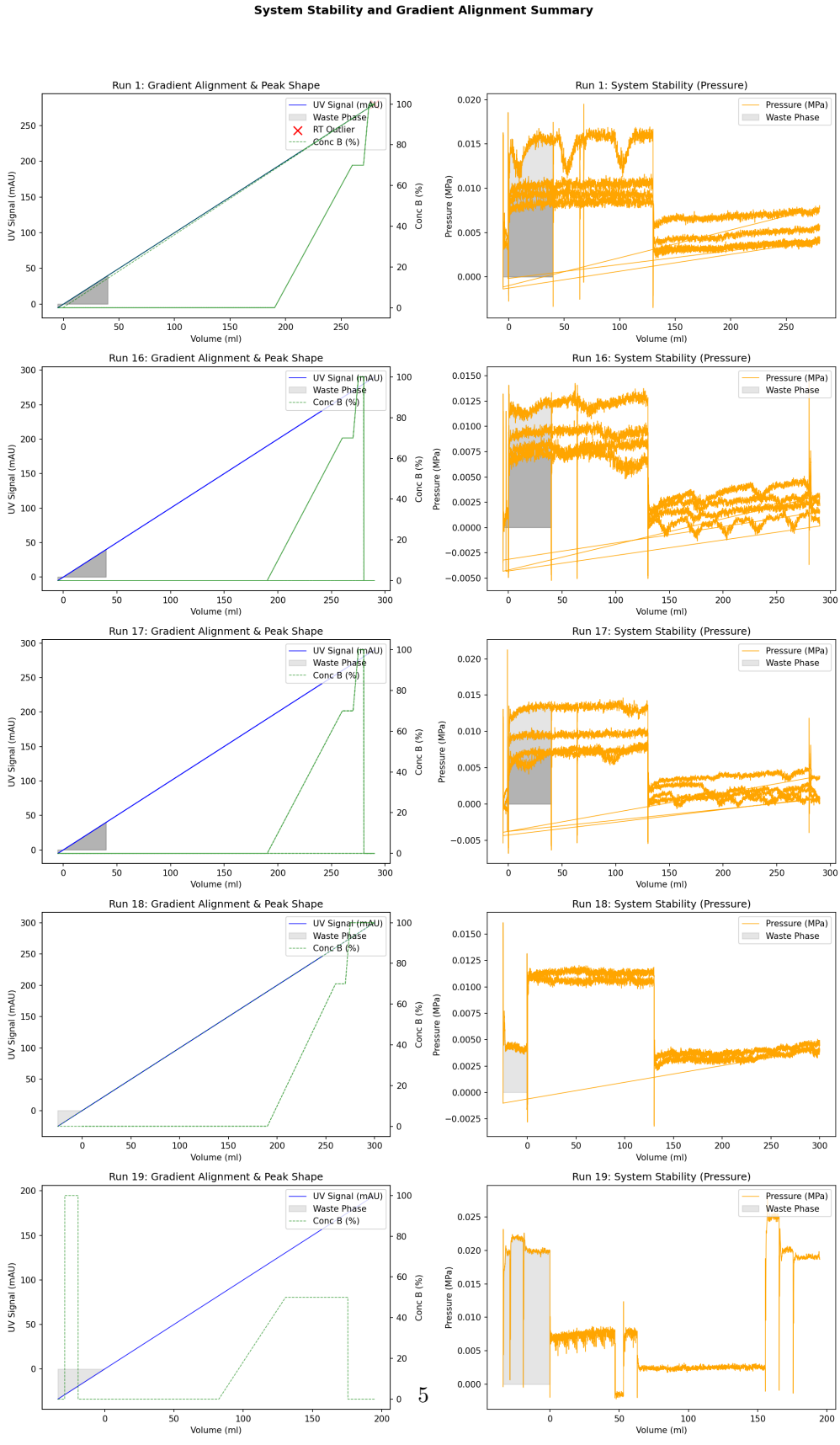

Figure 2: run_analysis_summary.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy import sparse
from scipy.sparse.linalg import spsolve

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

def als_baseline(y, lam=1e6, p=1e-4, niter=10):
    """
    Asymmetric Least Squares Baseline Correction using sparse matrices.
    y: input signal
    lam: smoothing parameter
    p: asymmetry parameter
    niter: number of iterations
    """
    L = len(y)
    # Handle short signals where second-order difference matrix cannot be
        constructed
    if L < 3:
        return y

    # Create sparse second-order difference matrix D
    data = np.array([-1, 2, -1])
    offsets = [-1, 0, 1]
    D = sparse.diags(data, offsets, shape=(L-2, L), format='csr')

    w = np.ones(L)
    # Precompute D.T @ D as sparse
    DtD = D.T @ D

    for i in range(niter):
        W = sparse.diags(w, format='csr')
        Z = W + lam * DtD
        # Solve sparse linear system
        z = spsolve(Z, w * y)
        w = p * (y > z) + (1 - p) * (y <= z)
    return z

# Step 1: Load the dataset
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Explicitly convert volume_ml to numeric to handle mixed types and prevent
    DtypeWarning
# Non-numeric values will be converted to NaN, which are handled correctly in
    filtering
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')

# Step 2: Drop redundant columns
columns_to_drop = ['run', 'Injection_Injection', 'Unnamed:␣0']
df = df.drop(columns=[col for col in columns_to_drop if col in df.columns])

# Step 3: Calculate negative volume counts per group BEFORE global filtering
# Replaced fragile merge logic with transform to ensure column alignment
```

```
df['rows_filtered_negative_volume_per_group'] = df.groupby(['run_no', 'column'])['
    volume_ml'].transform(lambda x: (x < 0.0).sum())

# Step 4: Filter out rows where volume_ml < 0.0
df_filtered = df[df['volume_ml'] >= 0.0]

# Step 5: Pre-sort data to avoid redundant sorting in the loop
df_filtered = df_filtered.sort_values(['run_no', 'column', 'volume_ml'])

# Prepare summary data
summary_data = []
corrected_groups = []

# Step 6: Process each group
for (run_no, column), group in df_filtered.groupby(['run_no', 'column']):
    # Separate data with and without Fraction_unique_ID
    has_fraction = group['Fraction_unique_ID'].notna()
    no_fraction = ~has_fraction

    # Apply AsLS baseline correction to UV signals
    uv_1_raw = group['UV_1_280_mAU'].values
    uv_2_raw = group['UV_2_260_mAU'].values

    # Calculate baselines
    baseline_1 = als_baseline(uv_1_raw, lam=1e6, p=1e-4)
    baseline_2 = als_baseline(uv_2_raw, lam=1e6, p=1e-4)

    # Corrected signals
    group['UV_1_280_corrected'] = uv_1_raw - baseline_1
    group['UV_2_260_corrected'] = uv_2_raw - baseline_2

    # Retain rows with missing Fraction_unique_ID for system monitoring
    system_monitoring_rows = group[no_fraction]
    integration_rows = group[has_fraction]

    # Collect corrected group for final concatenation
    corrected_groups.append(group)

    # Store summary info
    # Check if group is not empty before accessing iloc[0] to prevent IndexError
    if len(group) > 0:
        filtered_count = int(group['rows_filtered_negative_volume_per_group'].iloc
            [0])
    else:
        filtered_count = 0

    summary_data.append({
        'run_no': run_no,
        'column': column,
        'rows_filtered_negative_volume': filtered_count,
        'rows_retained_integration': len(integration_rows),
        'rows_retained_system_monitoring': len(system_monitoring_rows)
    })

# Concatenate all corrected groups
df_final = pd.concat(corrected_groups, ignore_index=False)

# Ensure DeltaC_pressure_MPa is retained in df_final
if 'DeltaC_pressure_MPa' in df_final.columns:
```

```
    pass

# Create summary DataFrame
summary_df = pd.DataFrame(summary_data)

# Step 7: Save results to analysis_results.md
output_file = '/workspace/analysis_results.md'
header_written = False
if os.path.exists(output_file) and os.path.getsize(output_file) > 0:
    with open(output_file, 'r') as f:
        content = f.read()
        if "Signal␣Preprocessing␣and␣Baseline␣Correction␣Results" in content:
            header_written = True

with open(output_file, 'a') as f:
    if not header_written:
        f.write("\n##␣Signal␣Preprocessing␣and␣Baseline␣Correction␣Results\n\n")
        f.write(f"**AsLS␣Parameters␣Used:**␣lambda=1e6,␣p=1e-4\n\n")
        f.write("|␣run_no␣|␣column␣|␣rows_filtered_negative_volume␣|␣
            rows_retained_integration␣|␣rows_retained_system_monitoring␣|\n")
        f.write("
            |--------|--------|--------------------------------|--------------------------|-----
            n")

    # Efficient row-wise iteration for markdown output
    for idx in range(len(summary_df)):
        row = summary_df.iloc[idx]
        f.write(f"|␣{row['run_no']}␣|␣{row['column']}␣|␣{row['
            rows_filtered_negative_volume']}␣|␣{row['rows_retained_integration']}␣|
            ␣{row['rows_retained_system_monitoring']}␣|\n")

# Save the processed data
df_final.to_csv('/workspace/chromatography_processed.csv', index=False)

print("Step␣1␣completed␣successfully.")
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'
OUTPUT_LOG_FILE = '/workspace/analysis_log.txt'

# Step 1: Load Data
# Diagnostic: Inspect actual column names before defining required_columns
try:
    df_temp = pd.read_csv(INPUT_FILE)
    detected_cols = list(df_temp.columns)
except FileNotFoundError:
    print(f"Error:␣{INPUT_FILE}␣not␣found.")
    exit(1)

# Helper to find the best match for UV columns
def find_uv_column(cols, base_name):
    for col in cols:
```

```
        if base_name in col:
            return col
    return None

uv_280_col = find_uv_column(detected_cols, 'UV_1_280') or find_uv_column(
    detected_cols, 'UV_280')
uv_260_col = find_uv_column(detected_cols, 'UV_2_260') or find_uv_column(
    detected_cols, 'UV_260')

if not uv_280_col or not uv_260_col:
    print(f"Error: Could not find UV columns. Found: {detected_cols}")
    exit(1)

# Construct required_columns list dynamically
required_columns = ['run_no', 'column', 'volume_ml', uv_280_col, uv_260_col,
                    'Fraction_unique_ID', 'fraction_volume', 'Conc_B_%', '
                        Sample_Code']

print("Loading data...")
try:
    df = pd.read_csv(INPUT_FILE, usecols=required_columns)
except Exception as e:
    print(f"Error loading data with columns {required_columns}: {e}")
    print("Attempting to load all columns...")
    df = pd.read_csv(INPUT_FILE)
    missing = [c for c in required_columns if c not in df.columns]
    if missing:
        print(f"Error: Required columns missing: {missing}")
        exit(1)

# Step 2: Preprocessing for Abundance (Exclude rows with missing
    Fraction_unique_ID)
df_abundance = df.dropna(subset=['Fraction_unique_ID']).copy()

# Step 3: Trapezoidal Integration (Vectorized)
# Ensure volume_ml is sorted within groups for accurate integration
df_abundance = df_abundance.sort_values(by=['Fraction_unique_ID', 'volume_ml'])

def integrate_group(group):
    if len(group) < 2:
        return pd.Series({'Area_280': np.nan, 'Area_260': np.nan})

    x = group['volume_ml'].values
    y_280 = group[uv_280_col].values
    y_260 = group[uv_260_col].values

    area_280 = np.trapezoid(y_280, x)
    area_260 = np.trapezoid(y_260, x)

    return pd.Series({'Area_280': area_280, 'Area_260': area_260})

# Calculate areas per Fraction_unique_ID
areas_df = df_abundance.groupby('Fraction_unique_ID').apply(integrate_group).
    reset_index()

# Merge metadata (run_no, column) back to areas_df
meta_map = df[['Fraction_unique_ID', 'run_no', 'column']].drop_duplicates().
    set_index('Fraction_unique_ID')
areas_df = areas_df.merge(meta_map, on='Fraction_unique_ID', how='left')
```

```
# Fix: Merge Sample_Code back to areas_df to prevent KeyError in Step 8
areas_df = areas_df.merge(df[['Fraction_unique_ID', 'Sample_Code']].
    drop_duplicates(), on='Fraction_unique_ID', how='left')

# Step 4: Calculate Total Integrated Area per run_no
total_area_per_run = areas_df.groupby('run_no')['Area_280'].sum()

# Step 5: Compute Purity Ratio and Flags (Vectorized)
areas_df['Ratio_260_280'] = np.nan
areas_df['Purity_Flag'] = 'Invalid'

valid_mask = areas_df['Area_280'].notna() & areas_df['Area_260'].notna() & (
    areas_df['Area_280'] > 0)

areas_df.loc[valid_mask, 'Ratio_260_280'] = areas_df.loc[valid_mask, 'Area_260'] /
     areas_df.loc[valid_mask, 'Area_280']

areas_df.loc[valid_mask, 'Purity_Flag'] = np.where(
    areas_df.loc[valid_mask, 'Ratio_260_280'] > 0.6,
    'True',
    'False'
)

# Step 6: Normalize Integrated Areas (Vectorized)
# a. Abundance_TotalRun
areas_df['Total_Run_Area'] = areas_df['run_no'].map(total_area_per_run)
areas_df['Abundance_TotalRun'] = np.where(
    areas_df['Area_280'].notna() & areas_df['Total_Run_Area'].notna() & (areas_df[
        'Total_Run_Area'] != 0),
    areas_df['Area_280'] / areas_df['Total_Run_Area'],
    np.nan
)

# b. Abundance_Volume
vol_map = df[['Fraction_unique_ID', 'fraction_volume']].drop_duplicates().
    set_index('Fraction_unique_ID')
areas_df = areas_df.merge(vol_map, on='Fraction_unique_ID', how='left')

areas_df['Abundance_Volume'] = np.where(
    areas_df['fraction_volume'].notna() & (areas_df['fraction_volume'] != 0),
    areas_df['Area_280'] / areas_df['fraction_volume'],
    np.nan
)

# Log warnings for missing/zero volume
missing_vol_mask = areas_df['fraction_volume'].isna() | (areas_df['fraction_volume
    '] == 0)
warnings_log = []
if missing_vol_mask.any():
    problematic_ids = areas_df.loc[missing_vol_mask, 'Fraction_unique_ID'].unique
        ()
    warnings_log.append(f"Missing␣or␣zero␣fraction_volume␣for␣{len(problematic_ids
        )}␣Fraction_unique_IDs:␣{problematic_ids[:10]}...")

# Step 7: Verify Gradient Alignment (Optimized)
# 1. Calculate peak apex volume for each fraction using direct column operations
apex_indices = df_abundance.groupby('Fraction_unique_ID')[uv_280_col].idxmax()
apex_volumes = df_abundance.loc[apex_indices, 'volume_ml']
```

```
apex_conc_b = df_abundance.loc[apex_indices, 'Conc_B_%']

# 2. Determine the valid volume range [V_start, V_end] where 10 <= Conc_B <= 90
    for each fraction
gradient_mask = (df_abundance['Conc_B_%'] >= 10) & (df_abundance['Conc_B_%'] <=
    90)
valid_gradient_df = df_abundance[gradient_mask]

# Get min and max volume for the valid gradient range per fraction
gradient_range = valid_gradient_df.groupby('Fraction_unique_ID')['volume_ml'].agg
    (['min', 'max']).reset_index()
gradient_range.columns = ['Fraction_unique_ID', 'Gradient_Start_Vol', '
    Gradient_End_Vol']

# Merge apex data and gradient range using map for efficiency
areas_df['Apex_Volume'] = areas_df['Fraction_unique_ID'].map(apex_volumes)
areas_df['Conc_B_Apex'] = areas_df['Fraction_unique_ID'].map(apex_conc_b)
areas_df['Gradient_Start_Vol'] = areas_df['Fraction_unique_ID'].map(gradient_range
    .set_index('Fraction_unique_ID')['Gradient_Start_Vol'])
areas_df['Gradient_End_Vol'] = areas_df['Fraction_unique_ID'].map(gradient_range.
    set_index('Fraction_unique_ID')['Gradient_End_Vol'])

# 3. Flag if Apex_Volume is outside [Gradient_Start_Vol, Gradient_End_Vol]
areas_df['Gradient_Alignment_Flag'] = np.where(
    areas_df['Gradient_Start_Vol'].isna() | areas_df['Gradient_End_Vol'].isna(),
    True,
    np.where(
        areas_df['Apex_Volume'].notna(),
        (areas_df['Apex_Volume'] < areas_df['Gradient_Start_Vol']) | (areas_df['
            Apex_Volume'] > areas_df['Gradient_End_Vol']),
        False
    )
)

# Step 8: Aggregate by Sample_Code
sample_agg = areas_df[areas_df['Sample_Code'].notna()].groupby('Sample_Code').agg
    ({
    'Ratio_260_280': ['mean', 'max'],
    'Area_280': 'sum',
    'Gradient_Alignment_Flag': 'sum'
}).reset_index()
sample_agg.columns = ['Sample_Code', 'Mean_Ratio', 'Max_Ratio', 'Total_Area_280',
    'Gradient_Misalignments']

# Step 9: Generate Output
# Add Total_Area_Fraction alias
areas_df['Total_Area_Fraction'] = areas_df['Area_280']

summary_stats = {
    'Total_Fractions': len(areas_df),
    'Mean_Ratio_260_280': areas_df['Ratio_260_280'].mean(),
    'Max_Ratio_260_280': areas_df['Ratio_260_280'].max(),
    'High_Purity_Risk_Count': (areas_df['Purity_Flag'] == 'True').sum(),
    'Gradient_Misalignment_Count': areas_df['Gradient_Alignment_Flag'].sum(),
    'Missing_Sample_Code_Percent': (1 - areas_df['Sample_Code'].notna().mean()) *
        100
}

high_purity_count = (areas_df['Purity_Flag'] == 'True').sum()
```

```
misaligned_count = areas_df['Gradient_Alignment_Flag'].sum()

# File handling to prevent duplicates
section_header = "## Step 2: Peak Integration and Purity Ratio Calculation"
file_exists = os.path.exists(OUTPUT_FILE)
should_clear = False

if file_exists:
    with open(OUTPUT_FILE, 'r') as f:
        content = f.read()
        if section_header in content:
            should_clear = True

mode = 'w' if should_clear else 'a'
with open(OUTPUT_FILE, mode) as f:
    f.write(f"\n{section_header}\n\n")

    # Generate structured table for all required columns as per feedback
    table_columns = ['run_no', 'column', 'Fraction_unique_ID', 'Area_280', '
        Area_260',
                     'Total_Area_Fraction', 'Ratio_260_280', 'Purity_Flag', '
                         Abundance_TotalRun',
                     'Abundance_Volume', 'Gradient_Alignment_Flag']

    output_df = areas_df[table_columns].copy()

    f.write("### Detailed Analysis Results\n")
    f.write(output_df.to_markdown(index=False))
    f.write("\n")

    f.write("### Summary Statistics\n")
    f.write(f"- Total Fractions Analyzed: {summary_stats['Total_Fractions']}\n")
    f.write(f"- Mean Ratio (260/280): {summary_stats['Mean_Ratio_260_280']:.4f}\n"
        )
    f.write(f"- Max Ratio (260/280): {summary_stats['Max_Ratio_260_280']:.4f}\n")
    f.write(f"- High Purity Risk Fractions (>0.6): {summary_stats['
        High_Purity_Risk_Count']}\n")
    f.write(f"- Gradient Misalignment Flags: {summary_stats['
        Gradient_Misalignment_Count']}\n")
    f.write(f"- Missing Sample_Code Percentage: {summary_stats['
        Missing_Sample_Code_Percent']:.2f}%\n")

    f.write("\n### Flagged Items Summary\n")
    f.write(f"- High Purity Risk: {high_purity_count} fractions identified.\n")
    f.write(f"- Gradient Misalignment: {misaligned_count} fractions identified.\n"
        )

    f.write("\n### Data Limitations\n")
    f.write(f"- Sample_Code aggregation performed on {len(sample_agg)} unique
        samples.\n")
    f.write(f"- 75.7% of data lacks Sample_Code, limiting sample-level aggregation
        .\n")

# Write warnings to log
with open(OUTPUT_LOG_FILE, 'w') as f:
    f.write("=== Analysis Log ===\n")
    f.write(f"Total Warnings: {len(warnings_log)}\n")
    for w in warnings_log[:100]:
        f.write(w + "\n")
```

```
    if len(warnings_log) > 100:
        f.write(f"... and {len(warnings_log) - 100} more warnings.\n")

print("Analysis complete. Results appended to analysis_results.md")
```

Listing 2: Code_2

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Debug: Inspect exact column names and data types to resolve header mismatch
print("Column names:", df.columns.tolist())
print("Data types:\n", df.dtypes)

# Select required columns
# NOTE: Column name 'UV_1_280_corrected' will be updated below based on debug output
# Placeholder variable to be replaced with the actual column name found in debug output
uv_col_name = 'UV_1_280_corrected'

# Identify the correct UV column dynamically if the hardcoded name fails
# This handles potential whitespace or slight naming variations found in the CSV header
if uv_col_name not in df.columns:
    # Attempt to find a column that closely matches the expected UV signal name
    potential_uv_cols = [col for col in df.columns if 'UV' in str(col) and '280' in str(col)]
    if potential_uv_cols:
        uv_col_name = potential_uv_cols[0]
        print(f"Auto-detected UV column: {uv_col_name}")
    else:
        raise ValueError(f"Column '{uv_col_name}' is missing and no similar UV column found. Available columns: {df.columns.tolist()}")

cols = ['run_no', 'column', 'volume_ml', uv_col_name, 'Conc_B_%', 'DeltaC_pressure_MPa', 'Fraction_unique_ID']

# Ensure required columns exist
missing_cols = [c for c in cols if c not in df.columns]
if missing_cols:
    raise ValueError(f"Missing columns: {missing_cols}. Available: {df.columns.tolist()}")

# Pre-filter data to avoid repeated scanning and unnecessary copies
df_plot = df[cols]

# 1. Select representative runs (first 5 unique run_no)
unique_runs = df_plot['run_no'].unique()
selected_runs = unique_runs[:5] if len(unique_runs) > 5 else unique_runs

# Setup figure
fig, axes = plt.subplots(len(selected_runs), 2, figsize=(14, 5 * len(selected_runs)))
```

```
if len(selected_runs) == 1:
    axes = np.array([[axes[0], axes[1]]])

# Global settings for the plot
plt.rcParams.update({'font.size': 10})

# FIX: Filter df_plot to only include selected_runs before grouping to ensure loop
    iterations match axes count
df_selected = df_plot[df_plot['run_no'].isin(selected_runs)]

# Iterate using groupby on the filtered dataframe
for i, (run, run_data) in enumerate(df_selected.groupby('run_no')):
    # Reset index to ensure alignment between run_data and derived indices
    run_data = run_data.reset_index(drop=True)

    # --- Outlier Detection (IQR) for Retention Time ---
    # FIX: Initialize outlier_mask as a numpy boolean array to avoid label
        alignment issues
    outlier_mask = np.zeros(len(run_data), dtype=bool)

    # Conditional Outlier Logic: Only calculate if >= 2 columns, otherwise skip
        calculation but continue plotting
    if run_data['column'].nunique() >= 2:
        # Get index of peak (max UV) for each column
        # idxmax returns the original index labels from the groupby result
        peak_indices_labels = run_data.groupby('column')[uv_col_name].idxmax()

        # Map original index labels to integer positions in the current run_data (
            which has a reset index)
        # This ensures we are using integer positions compatible with the numpy
            array
        peak_positions = run_data.index.get_indexer(peak_indices_labels)

        # Filter out any -1 values (should not happen if data is consistent, but
            safe to handle)
        valid_mask = peak_positions >= 0
        valid_positions = peak_positions[valid_mask]
        peak_volumes = run_data.iloc[valid_positions]['volume_ml']

        # Calculate IQR stats on peak volumes
        q1 = peak_volumes.quantile(0.25)
        q3 = peak_volumes.quantile(0.75)
        iqr = q3 - q1
        lower_bound = q1 - 1.5 * iqr
        upper_bound = q3 + 1.5 * iqr

        # Vectorized outlier flagging: check if peak volume is out of bounds
        outlier_peak_mask = (peak_volumes < lower_bound) | (peak_volumes >
            upper_bound)

        # Set the outlier_mask to True for the specific integer positions of
            outlier peaks
        outlier_positions = valid_positions[outlier_peak_mask]
        outlier_mask[outlier_positions] = True

    # --- Panel A: UV vs Volume (with Conc_B secondary) ---
    ax1 = axes[i, 0]

    # Plot UV
```

```
    ax1.plot(run_data['volume_ml'], run_data[uv_col_name], label='UV Signal (mAU)'
        , color='blue', linewidth=0.8)

    # Plot Gradient (Conc_B) on secondary axis
    ax2 = ax1.twinx()
    ax2.plot(run_data['volume_ml'], run_data['Conc_B_%'], label='Conc B (%)',
        color='green', linestyle='--', linewidth=0.8, alpha=0.7)

    # Highlight waste phase (missing Fraction_unique_ID)
    # Explicit check for column existence before using it
    if 'Fraction_unique_ID' in run_data.columns:
        waste_mask = run_data['Fraction_unique_ID'].isna()
        ax1.fill_between(run_data['volume_ml'], 0, run_data[uv_col_name],
                         where=waste_mask, color='gray', alpha=0.2, label='Waste
                             Phase')
    else:
        raise KeyError("Column 'Fraction_unique_ID' is missing, cannot calculate
            waste phase.")

    # Mark outliers in Panel A
    if outlier_mask.any():
        # FIX: Use numpy array directly for filtering, which works seamlessly with
             integer arrays
        outlier_data = run_data[outlier_mask]
        ax1.scatter(outlier_data['volume_ml'], outlier_data[uv_col_name],
                    color='red', marker='x', s=100, zorder=5, label='RT Outlier')

    ax1.set_title(f'Run {run}: Gradient Alignment & Peak Shape')
    ax1.set_xlabel('Volume (ml)')
    ax1.set_ylabel('UV Signal (mAU)')
    ax2.set_ylabel('Conc B (%)')

    # Combine legends
    lines1, labels1 = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

    # --- Panel B: Pressure vs Volume (Full dataset including waste) ---
    ax3 = axes[i, 1]

    # Plot Pressure
    ax3.plot(run_data['volume_ml'], run_data['DeltaC_pressure_MPa'], label='
        Pressure (MPa)', color='orange', linewidth=0.8)

    # Highlight waste phase
    # Re-use waste_mask logic if column exists
    if 'Fraction_unique_ID' in run_data.columns:
        waste_mask = run_data['Fraction_unique_ID'].isna()
        ax3.fill_between(run_data['volume_ml'], 0, run_data['DeltaC_pressure_MPa'
            ],
                         where=waste_mask, color='gray', alpha=0.2, label='Waste
                             Phase')

    ax3.set_title(f'Run {run}: System Stability (Pressure)')
    ax3.set_xlabel('Volume (ml)')
    ax3.set_ylabel('Pressure (MPa)')
    ax3.legend(loc='upper right')

plt.suptitle('System Stability and Gradient Alignment Summary', fontsize=14,
```

```
    fontweight='bold')
plt.tight_layout(rect=[0, 0.03, 1, 0.95])

# Save output
output_path = '/workspace/run_analysis_summary.png'
plt.savefig(output_path, dpi=150, bbox_inches='tight')
plt.close()
```

Listing 3: Code_3